



南開大學
Nankai University

南开大学
编译系统原理 实验报告

Lab 1: 编译器、LLVM IR 与
RISC-V 汇编

成员 A (LLVM IR): 李培涛 / 2411041

成员 B (RISC-V): 梁家瑞 / 2411046

专业: 计算机科学与技术 / 信息安全

2026 年 10 月 1 日

目 录

实验分工	3
成员 A 独立报告 (LLVM IR)	4
1 摘要	4
2 引言	4
2.1 实验背景	4
2.2 实验范围与分工	4
2.3 内容安排	4
3 第一部分：语言处理系统完整工作过程	4
3.1 研究对象与阶段总览	4
3.2 预处理器做了什么	5
3.3 编译器做了什么	5
3.4 汇编器做了什么	6
3.5 链接器做了什么	6
3.6 端到端过程的验证边界	6
4 第二部分：LLVM IR 与汇编语言实验	6
4.1 实验环境	6
4.2 示例程序逻辑	7
4.3 源程序	7
4.4 预处理与 LLVM IR 生成	8
4.5 LLVM IR 结构分析	10
4.6 对比实验与边界实验	12
5 第三部分：MLIR 中端处理过程（进阶要求）	13
5.1 任务目标与层级组织	13
5.2 VecAdd 的逐层 lowering 观察点	13
5.3 命令与结果记录	14
5.4 进阶实验的验证边界	14
6 结果与讨论	15
6.1 已获得的结果	15
6.2 实验条件与验证边界	15
6.3 验证边界	15
7 结论	15
成员 A 附录：复现实验命令	17

成员 B 独立报告（RISC-V 与汇编语言）	18
8 摘要	18
9 引言	18
9.1 实验背景	18
9.2 实验范围与分工	18
9.3 内容安排	18
10 第一部分：语言处理系统完整工作过程	18
10.1 研究对象与阶段总览	18
10.2 预处理器做了什么	19
10.3 编译器做了什么	19
10.4 汇编器做了什么	19
10.5 链接器做了什么	19
10.6 端到端过程的验证边界	19
11 第二部分：RISC-V 与汇编语言实验	19
11.1 实验环境与文件	19
11.2 生成并检查 RISC-V 汇编	20
11.3 函数序言、栈帧和 ABI	20
11.4 数组寻址与内存访问	20
11.5 循环、分支与算术指令	21
11.6 函数调用和运行库接口	21
11.7 汇编、链接与运行	21
11.8 实验结果与分析	22
12 第三部分：MLIR 中端处理过程（进阶要求）	22
12.1 任务目标与层级组织	22
12.2 VecAdd 的逐层 lowering 观察点	22
12.3 命令与结果记录	22
12.4 进阶实验的验证边界	23
13 结果与讨论	23
13.1 已获得的结果	23
13.2 结果解释与验证边界	23
14 结论	23
成员 B 附录：复现实验命令	25
参考文献	26

实验分工

本实验分为 LLVM IR 前端分析和 RISC-V 后端分析两部分。成员 A 李培涛（2411041）负责预处理、LLVM IR 生成、LLVM IR 验证和 IR 结构分析；成员 B 梁家瑞（2411046）负责 LLVM IR 到 RISC-V 汇编的降低、汇编语言阅读、汇编与链接、ELF 检查和目标程序执行分析。两部分使用相同的 SysY 源程序、运行库接口和实验环境，并共同分析 MLIR/AscendNPU 的静态 lowering 过程。

成员 A 独立报告（LLVM IR）

1 摘要

本实验以一个覆盖多种 SysY 语言特性的示例程序为对象，重点研究从源程序到 LLVM 中间表示（LLVM IR）的转换过程。实验首先使用 Clang 的预处理选项观察运行库声明如何注入源程序；随后使用 Clang 生成面向目标平台的 LLVM IR，并用 LLVM 验证器检查模块结构是否合法。报告进一步从全局对象、局部变量、数组寻址、循环、条件分支、函数调用和运行库声明等角度，将源代码与 LLVM IR 指令逐项对应。

示例程序先将数组中的每个元素乘以 2，再求正数之和与正数个数，最后根据整数平均值返回分类结果。实验记录中的主机参考程序输出为 28 3 1，LLVM IR 验证通过。本部分分析预处理、LLVM IR 生成、LLVM IR 验证和 IR 结构；RISC-V 汇编、汇编器和目标程序结果见后文的后端部分。

关键词：编译器；预处理器；LLVM IR；SysY；中间表示；控制流；数组寻址

2 引言

2.1 实验背景

编译器将高级语言程序转换为目标机器可以执行的程序。这个过程不是单次文本替换，而是由预处理、词法分析、语法分析、语义分析、中间表示构造、优化、代码生成、汇编和链接等阶段组成。LLVM 将中间表示作为编译器前端和后端之间的接口：前端负责把源程序的结构和语义编码到 LLVM IR，后续优化与代码生成则以 IR 为输入。LLVM IR 的指令、类型、基本块和控制流关系可以直接观察，因此适合用来理解源程序的语义如何被编译器表示 [5, 4]。

本实验使用 SysY 子集程序作为输入。SysY 的语法比完整 C 语言更小，但包含常量、变量、数组、表达式、条件、循环、函数和运行库调用等编译器实验需要的核心结构 [6]。示例程序不直接包含主机输入输出头文件，而是通过 `sysy_runtime.h` 声明 `putint` 和 `putch`，从而把语言特性和运行库实现分开。

2.2 实验范围与分工

本部分覆盖预处理器观察、Clang 生成 IR、`opt` 验证 IR、IR 指令阅读和对比实验。RISC-V 汇编、汇编器、链接器和目标程序分析在后文展开。两部分均以实际生成的中间文件、目标文件和运行日志为依据。

2.3 内容安排

第 1 部分说明预处理器、编译器各阶段、汇编器和链接器的工作过程；第 2 部分记录 SysY 示例、LLVM IR 生成与验证、IR 结构分析和对比实验；第 3 部分记录 MLIR 中端 `VecAdd` 的逐层 lowering 观察；最后给出结果讨论、结论和复现命令。

3 第一部分：语言处理系统完整工作过程

3.1 研究对象与阶段总览

本部分以 Clang/LLVM 工具链为研究对象，说明一个 C/SysY 风格源程序如何经过预处理、编译、汇编和链接，最终形成可执行目标。命令行工具可以把完整过程拆开观察：`clang -E` 停在预处理结果，`clang -S -emit-llvm` 输出 LLVM IR，`clang -S` 输出汇编，汇编器把汇编文本转换为目标文件，链接器再把目标文件和运行库组合为可执行文件。

表 1: 语言处理系统各阶段及其主要产物

阶段	主要功能	典型产物
预处理	展开头文件、宏和条件编译，形成编译器前端的输入	.i
词法分析	将字符流切分为关键字、标识符、常量和运算符等记号	token 流
语法分析	根据文法构造抽象语法树并报告语法错误	AST
语义分析	检查类型、作用域、返回值和控制流约束	带类型的 AST
IR 生成与优化	把 AST 转换为中间表示并进行目标无关优化	LLVM IR
代码生成	结合目标架构选择指令、寄存器和栈布局	汇编文本
汇编	编码指令、建立符号表并生成重定位信息	.o
链接	合并目标文件、解析外部符号并完成重定位	ELF/可执行文件

3.2 预处理器做了什么

预处理器在编译器正式解析前处理文本级指令。头文件包含会把声明和宏定义插入当前翻译单元，宏展开会替换标识符，条件编译则根据宏环境保留或删除代码。预处理阶段通常不建立完整的类型和控制流语义，因此 `clang -E` 的输出仍然是源代码文本，只是已经完成了这些文本变换。

本实验使用 `-include sysy_runtime.h` 强制注入 `putint` 和 `putch` 的声明。预处理结果用于确认运行库接口确实进入了翻译单元；声明是否匹配调用参数，则由后续语义分析和类型检查负责。

Listing 1: 观察预处理阶段

```
1 clang -E -std=c11 -include sysy_runtime.h \
2   SysY_example.c -o build/SysY_example.i
```

3.3 编译器做了什么

3.3.1 词法分析与语法分析

词法分析把字符序列转换为记号，例如 `int`、函数名、整数常量、数组下标和关系运算符。语法分析按照 SysY/C 文法把记号组织成声明、表达式、语句、函数定义和程序整体的抽象语法树。括号不匹配、缺少分号或语句结构不符合文法的错误通常在这两个阶段被报告。

3.3.2 语义分析

语义分析检查标识符是否已声明、变量和函数的类型是否匹配、数组下标是否用于可索引对象、函数返回值是否满足声明，以及整数除法等运算是否满足语言规则。作用域和符号表也在此阶段建立。预处理器不会完成这些检查，因此预处理文件可以生成并不代表源程序已经语义正确。

3.3.3 LLVM IR 生成、验证和优化

Clang 前端把带类型的 AST 转换为 LLVM IR。全局变量、函数参数、基本块、内存访问和控制流都被编码为 IR 对象和指令；`opt -passes=verify` 检查模块结构是否满足 LLVM IR 的约束。优化器可以进行常量传播、死代码删除、控制流简化和内存到寄存器提升，但必须保持程序语义 [5]。

3.3.4 目标代码生成

后端根据目标三元组和 ABI 把 LLVM IR 降低为目标架构的指令。这个阶段需要决定寄存器分配、栈帧布局、调用约定、指令选择和分支形式。本报告分别分析 LLVM IR 和 RISC-V 汇编，并以实际生成的文件和运行结果作为结论依据。

3.4 汇编器做了什么

汇编器读取汇编文本，识别指令和伪指令，把指令操作数编码为机器码，同时建立局部和全局符号表。对于尚不能在当前文件内确定的地址，汇编器在目标文件中留下重定位项；外部函数调用因此可以先生成目标文件，待链接器统一解析。典型命令如下，实际架构选项以小组后端实验使用的工具链为准：

Listing 2: 汇编器阶段命令

```
1 riscv64-unknown-elf-as -march=rv64gc -mabi=lp64d \  
2   build/SysY_example.s -o build/SysY_example.o  
3 readelf -h -S -s build/SysY_example.o
```

3.5 链接器做了什么

链接器把一个或多个目标文件、启动代码和运行库合并，按照链接脚本安排代码段、只读数据段、已初始化数据段和未初始化数据段。它解析 `putint`、`putch` 等外部符号，计算跨文件调用地址并应用重定位，最后写出 ELF 或其他目标格式的可执行文件。若符号缺失、ABI 不兼容或链接脚本不匹配，错误会在此阶段出现。

Listing 3: 链接器阶段命令

```
1 riscv64-unknown-elf-gcc -march=rv64gc -mabi=lp64d \  
2   build/SysY_example.o sysy_runtime.c \  
3   -o build/SysY_example.elf  
4 readelf -h -S -s build/SysY_example.elf
```

3.6 端到端过程的验证边界

每个阶段的成功含义不同：预处理成功只表示文本变换完成，IR 验证通过只表示 IR 结构合法，汇编成功只表示当前汇编文本可以编码，链接成功只表示目标文件和外部符号能够组合。只有在目标环境中执行并得到预期输出，才能把整个流水线的行为验证闭环。本文对 LLVM IR 阶段给出完整分析，对 RISC-V 后端只保留流程说明和分工边界。

4 第二部分：LLVM IR 与汇编语言实验

4.1 实验环境

实验在 WSL 环境中完成，仓库已有记录使用 Clang/LLVM 18.1.3。生成 LLVM IR 的源文件和辅助文件位于 `lab1/SysY_example/`：

表 2: 实验文件及作用

文件	作用
SysY_example.c	SysY 子集示例源程序
sysy_runtime.h	putint、putch 的函数声明
sysy_runtime.c	输出函数的最小适配实现
SysY_example.ll	Clang 生成的 LLVM IR 文本模块
SysY_example_exercises.md	IR 阅读和对比实验清单
build_and_verify.sh	自动执行主机编译、IR 生成和 IR 验证的脚本入口

本次使用的 `clang --version`、`opt --version` 和 `llc --version` 输出已保存到 `build/report_results/tools_versions.txt`; RISC-V 运行链路的完整日志保存在同一目录。

4.2 示例程序逻辑

程序定义常量 `scale_factor=2` 和全局变量 `positive_count`。`main` 创建数组 `[3, -2, 7, 4, -1]`，然后执行以下三个步骤：

1. `scale_array` 使用 `while` 循环将每个元素乘以 2，得到 `[6, -4, 14, 8, -2]`。
2. `sum_positive` 遍历数组，累加正数 $6 + 14 + 8 = 28$ ，并将正数个数记录为 3。
3. `classify_average` 计算整数平均值 $28/3 = 9$ 和余数 1。平均值不小于 5 且余数非负，因此返回分类值 1。

程序最后通过 `putint` 和 `putch` 输出三个整数和换行，预期输出为：

Listing 4: 示例程序的预期输出

```
1 28 3 1
```

4.3 源程序

下面的源程序覆盖了常量、全局变量、一维数组、数组形参、算术运算、关系运算、逻辑运算、条件分支、循环、`int/void` 函数、函数调用和运行库调用。

Listing 5: SysY 子集示例程序

```

1 const int scale_factor = 2;
2 int positive_count = 0;
3
4 void scale_array(int values[], int length, int factor) {
5     int i = 0;
6     while (i < length) {
7         values[i] = values[i] * factor;
8         i = i + 1;
9     }
10 }
11
12 int sum_positive(int values[], int length) {
13     int sum = 0;
14     int i = 0;
15     positive_count = 0;
16

```

```

17 while (i < length) {
18     if (values[i] > 0) {
19         sum = sum + values[i];
20         positive_count = positive_count + 1;
21     }
22     i = i + 1;
23 }
24
25 return sum;
26 }
27
28 int classify_average(int sum, int count) {
29     if (count == 0) {
30         return 0;
31     }
32
33     int average = sum / count;
34     int remainder = sum % count;
35
36     if (average >= 5 && remainder >= 0) {
37         return 1;
38     }
39     if (average < 0 || !(average >= 5)) {
40         return -1;
41     }
42     return 1;
43 }
44
45 int main() {
46     int values[5] = {3, -2, 7, 4, -1};
47
48     scale_array(values, 5, scale_factor);
49     int sum = sum_positive(values, 5);
50     int category = classify_average(sum, positive_count);
51
52     putint(sum);
53     putch(32);
54     putint(positive_count);
55     putch(32);
56     putint(category);
57     putch(10);
58     return 0;
59 }

```

4.4 预处理与 LLVM IR 生成

4.4.1 预处理器观察

预处理器负责处理头文件包含、宏和条件编译，并输出经过文本预处理的源程序。虽然示例程序没有直接写 `#include <stdio.h>`，编译命令使用 `-include sysy_runtime.h` 强制注入运行库声明：

Listing 6: 生成预处理结果

```

1 cd "/mnt/d/code_warehouse/Curriculum/Compiler Systems/lab1/SysY_example"
2 clang -E -std=c11 -include sysy_runtime.h \

```

```
3 SysY_example.c -o build/SysY_example.i
```

在 build/SysY_example.i 中实际找到以下声明:

Listing 7: 强制包含的运行库声明

```
1 void putint(int value);
2 void putch(int value);
```

这一步只改变编译器前端看到的源文本, 不负责检查程序的控制流和类型语义。函数是否被正确调用, 要到后续的语义分析和 IR 生成阶段才能体现。

4.4.2 生成主机参考结果

先用主机 Clang 编译源程序和最小运行库适配层, 建立可用于对照的输出:

Listing 8: 生成主机参考程序

```
1 clang -std=c11 -Wall -Wextra -Werror \
2   -include sysy_runtime.h SysY_example.c sysy_runtime.c \
3   -o build/SysY_example.host
4 ./build/SysY_example.host
```

仓库已有验证记录中的主机输出为 28 3 1。该结果用于检查源程序的预期语义, 不能替代对 LLVM IR 结构的检查。

4.4.3 生成 LLVM IR

使用 Clang 的 -S -emit-llvm 选项将编译流程停在 LLVM IR 文本阶段:

Listing 9: 生成 LLVM IR

```
1 clang -target riscv64-unknown-elf -march=rv64gc -mabi=lp64d \
2   -std=c11 -O0 -S -emit-llvm -include sysy_runtime.h \
3   SysY_example.c -o SysY_example.ll
```

这里的 -O0 用于保留更接近源程序的局部变量和基本块, 适合初次阅读。生成模块的开头包含目标数据布局和目标三元组; 这些信息说明 IR 将按照目标平台的指针宽度、ABI 和数据布局解释, 但本报告不展开后续汇编生成。

Listing 10: LLVM IR 模块头和全局对象

```
1 ; ModuleID = 'SysY_example.c'
2 source_filename = "SysY_example.c"
3 target triple = "riscv64-unknown-unknown-elf"
4
5 @scale_factor = dso_local constant i32 2, align 4
6 @positive_count = dso_local global i32 0, align 4
7 @__const.main.values = private unnamed_addr constant [5 x i32]
8   [i32 3, i32 -2, i32 7, i32 4, i32 -1], align 4
```

常量 scale_factor 被放在只读全局对象中; positive_count 是可写全局对象; main 中的数组初值被保存为一个私有常量数组, 进入函数后再复制到局部数组。

4.4.4 验证 LLVM IR

生成文本 IR 后, 用 LLVM 验证器检查模块是否满足 IR 的结构约束:

Listing 11: 验证 LLVM IR

```
1 opt -passes=verify SysY_example.ll -disable-output
```

命令没有标准输出且返回码为 0 时, 表示验证器没有发现结构错误。验证器只检查 IR 的合法性, 不证明程序在所有输入上的算法语义正确; 语义检查仍需要结合源程序、边界输入和运行结果。

4.5 LLVM IR 结构分析

4.5.1 函数参数与局部变量

在 -O0 生成的 IR 中, 函数参数和局部变量通常先通过 `alloca` 建立栈槽, 再用 `store` 初始化、用 `load` 读取。例如 `scale_array` 的开头可以概括为:

Listing 12: `scale_array` 的参数与局部变量初始化

```
1 define dso_local void @scale_array(ptr noundef %0,
2   i32 noundef signext %1, i32 noundef signext %2) {
3   %4 = alloca ptr, align 8
4   %5 = alloca i32, align 4
5   %6 = alloca i32, align 4
6   %7 = alloca i32, align 4
7   store ptr %0, ptr %4, align 8
8   store i32 %1, ptr %5, align 4
9   store i32 %2, ptr %6, align 4
10  store i32 0, ptr %7, align 4
11  br label %8
12 }
```

四个栈槽分别保存数组首地址、数组长度、缩放因子和循环变量 `i`。`alloca` 只分配地址空间; 实际的值传递通过 `load` 和 `store` 完成。优化级别提高后, LLVM 可能把部分变量提升到寄存器或合并基本块, 因此 -O0 与 -O2 的 IR 形态会明显不同。

4.5.2 while 循环和条件分支

`scale_array` 的循环条件被表示为比较指令和条件分支:

Listing 13: `scale_array` 的循环条件

```
1 8:
2   %9 = load i32, ptr %7, align 4
3   %10 = load i32, ptr %5, align 4
4   %11 = icmp slt i32 %9, %10
5   br i1 %11, label %12, label %26
```

当 %11 为真时进入循环体 %12, 否则跳到 %26 返回。循环体最后通过无条件分支回到 %8, 构成控制流回边。源程序中的 `while (i < length)` 因此被拆成了“循环头—循环体—结束块”三个基本结构。

`sum_positive` 还包含嵌套的 `if`。循环头 %7 检查 `i < length`, 元素判断块 %11 检查 `values[i] > 0`, 正数路径 %18 更新 `sum` 和 `positive_count`, 汇合块 %28 递增 `i` 并回到循环头。

4.5.3 数组寻址与 getelementptr

数组形参 `int values[]` 在 LLVM IR 中表现为指针参数, 因为函数接收的是数组首元素地址而不是整个数组副本。访问 `values[i]` 时, IR 先把 32 位下标扩展为索引宽度, 再使用 `getelementptr` 计算元素地址:

Listing 14: 数组元素寻址

```
1 %13 = load i32, ptr %6, align 4
2 %14 = sext i32 %13 to i64
3 %15 = getelementptr inbounds i32, ptr %12, i64 %14
4 %16 = load i32, ptr %15, align 4
```

`getelementptr` 不读取内存, 它只根据元素类型和索引计算地址。真正读取元素的是 `load`。写回元素时使用同样的地址计算过程, 再由 `store` 将乘法结果写入该地址。由于 `i32` 占 4 字节, 目标后端会据此计算元素偏移; 该报告只分析其 LLVM IR 来源。

4.5.4 算术、关系和逻辑运算

示例程序中不同运算在 LLVM IR 中有明确对应:

表 3: 源代码运算与 LLVM IR 指令对应

源代码运算	LLVM IR 表达
<code>+</code> 、 <code>-</code> 、 <code>*</code>	<code>add</code> 、 <code>sub</code> 、 <code>mul</code>
<code>/</code> 、 <code>%</code>	<code>sdiv</code> 、 <code>srem</code>
<code><</code> 、 <code>></code> 、 <code>==</code> 、 <code>>=</code>	<code>icmp slt</code> 、 <code>icmp sgt</code> 、 <code>icmp eq</code> 、 <code>icmp sge</code>
<code>&&</code>	前一个条件成立后进入后一个条件的基本块
<code> </code> 、 <code>!</code>	通过多个比较结果和分支块表达短路逻辑

逻辑运算不一定表现为一个名为 `and` 或 `or` 的算术指令。对于含有短路语义的表达式, Clang 会使用多个基本块和 `br` 指令保证求值顺序。`classify_average` 中的 `count == 0` 保护分支也确保除法和取余只在计数非零时执行。

4.5.5 函数调用、返回值和运行库声明

LLVM IR 使用 `define` 定义函数, 使用 `call` 调用函数, 使用 `ret` 返回。`main` 的调用链为:

Listing 15: main 中的函数调用和运行库调用

```
1 call void @scale_array(ptr noundef %5, i32 noundef signext 5,
2   i32 noundef signext 2)
3 %7 = call signext i32 @sum_positive(ptr noundef %6,
4   i32 noundef signext 5)
5 %10 = call signext i32 @classify_average(i32 noundef signext %8,
6   i32 noundef signext %9)
7 call void @putint(i32 noundef signext %11)
8 call void @putch(i32 noundef signext 32)
```

`putint` 和 `putch` 在当前模块中只有声明, 没有函数体:

Listing 16: 运行库外部声明

```
1 declare dso_local void @putint(i32 noundef signext)
2 declare dso_local void @putch(i32 noundef signext)
```

这体现了编译与链接之间的接口边界。LLVM IR 可以引用外部函数，但外部函数的最终实现由运行库在后续目标构建阶段提供。

4.6 对比实验与边界实验

4.6.1 修改平均值阈值

将源程序中的 `average >= 5` 修改为 `average >= 10`，重新生成 IR：

Listing 17: 重新生成修改后的 LLVM IR

```
1 clang -target riscv64-unknown-elf -march=rv64gc -mabi=lp64d \
2   -std=c11 -O0 -S -emit-llvm -include sysy_runtime.h \
3   SysY_example.c -o build/SysY_example.threshold10.ll
```

本次比较了 `classify_average` 中 `icmp sge` 的立即数。阈值从 5 改为 10 后，比较指令的常量由 5 变为 10；输入数组不变时平均值为 9，分类输出由 28 3 1 变为 28 3 -1，结果保存在 `threshold10.host.out`。

4.6.2 优化级别对比

分别以 `-O0` 和 `-O2` 生成 IR：

Listing 18: 比较不同优化级别的 LLVM IR

```
1 clang -target riscv64-unknown-elf -march=rv64gc -mabi=lp64d \
2   -O0 -S -emit-llvm -include sysy_runtime.h \
3   SysY_example.c -o build/SysY_example.O0.ll
4
5 clang -target riscv64-unknown-elf -march=rv64gc -mabi=lp64d \
6   -O2 -S -emit-llvm -include sysy_runtime.h \
7   SysY_example.c -o build/SysY_example.O2.ll
8
9 diff -u build/SysY_example.O0.ll build/SysY_example.O2.ll \
10  > build/SysY_example.O0-O2.diff || true
```

本次记录了三类差异：局部变量的 `alloca/load/store` 数量、循环基本块结构，以及函数属性、常量传播和内存访问变化。两个优化级别的语义输出保持一致，但 IR 结构发生了变化，具体差异保存在 `SysY_example.O0-O2.diff`。

4.6.3 零长度输入与除零保护

零长度变体将 `main` 中的输入长度设为 0。生成的 IR 保留 `count == 0` 分支，并在该分支直接返回 0，未执行 `sdiv` 或 `srem`；程序输出为 0 0 0，结果保存在 `zero.host.out` 和 `SysY_example.zero.ll`。该实验验证的是源程序显式写出的保护逻辑，而不是 LLVM 自动修复除零错误。

4.6.4 实验记录表

本次实验产物保存在 `SysY_example/build/report_results/`。表 4 记录这些文件对应的实际结果。

表 4: LLVM IR 对比实验记录

实验	应观察的 IR 变化	已保存的结果文件
阈值 5 → 10	icmp sge 立即数由 5 变为 10, 分类输出由 28 3 1 变为 28 3 -1; 见 threshold10.	host.out、SysY_example.
-00 与 -02	alloca: 17 → 0, load: 36 → 3, store: 25 → 4, 基本块: 17 → 10; 见 SysY_example.	threshold10.ll
count=0	输出 0 0 0, IR 保留除零保护分支; 见 zero.host.out、SysY_example.zero.ll	
预处理结果	SysY_example.i 第 12-13 行出现 putint、putch 声明	

5 第三部分：MLIR 中端处理过程（进阶要求）

5.1 任务目标与层级组织

LLVM IR 通常作为一个较低层级的统一中间表示，而 MLIR 允许同一程序在多个方言中逐步降低 [3]。VecAdd 示例可以先使用较高层的张量或线性代数操作表达向量加法，再依次降低到循环、向量指令、LLVM 方言和目标相关表示。AscendNPU IR 属于 MLIR 生态中的硬件抽象层。本次已在 WSL Ubuntu 24.04 中构建 AscendNPU-IR 2.0.0 (LLVM 19.1.7)，并保存通用 MLIR 与 HIVM 静态 lowering 产物；没有把未具备 CANN/昇腾设备条件的端到端编译写成已完成结果。

5.2 VecAdd 的逐层 lowering 观察点

本次将每次已执行转换的输入和输出单独保存，并比较操作名称、类型、显式控制流和内存访问的变化。观察顺序如下：

1. 高层方言：以 `linalg.generic`、`tensor` 或课程 VecAdd 方言表达逐元素加法；
2. 循环方言：转换为 `affine.for` 或 `scf.for`，显式出现迭代变量和边界；
3. 向量方言：使用 `vector.transfer_read`、`vector.add` 和 `vector.transfer_write` 表达批量数据操作；
4. LLVM 方言：将内存、指针、控制流和函数签名降低为 LLVM dialect 操作；
5. 目标相关阶段：由目标后端或 AscendNPU 相关方言继续完成硬件映射，具体结果取决于课程提供的官方工具链。

Listing 19: VecAdd 高层方言的观察示意

```

1 func.func @vec_add(%a: tensor<1024xf32>,
2                   %b: tensor<1024xf32>) -> tensor<1024xf32> {
3   %c = linalg.generic
4     {indexing_maps = [...], iterator_types = ["parallel"]}
5     ins(%a, %b : tensor<1024xf32>, tensor<1024xf32>)
6     outs(%a : tensor<1024xf32>) {

```

```

7   ^bb0(%x: f32, %y: f32, %out: f32):
8       %sum = arith.addf %x, %y : f32
9       linalg.yield %sum : f32
10  } -> tensor<1024xf32>
11  return %c : tensor<1024xf32>
12  }

```

上面代码用于说明需要关注的操作层级, 实际输入采用官方 VecAdd 示例及其对应版本。分析重点是确认每一步 lowering 是否保持输入输出形状、元素类型、迭代范围和逐元素加法语义。

5.3 命令与结果记录

以下命令展示本次实际运行的通用和 AscendNPU lowering。不同版本的 pass 名称和依赖可能不同, 执行前应先运行工具的 `--help`, 并记录实际版本。

Listing 20: MLIR/AscendNPU lowering 实际执行命令

```

1 bishengir-opt --version
2 bishengir-opt ascendnpusmokeyinput.mlir \
3   --convert-arith-to-llvm \
4   --convert-func-to-llvm \
5   -o ascendnpusmokeyllvm.mlir
6 bishengir-opt ascendnpuhivmconversioninput.mlir \
7   --split-input-file \
8   --convert-hivm-to-llvm \
9   -o ascendnpuhivmtollvm.mlir

```

表 5: MLIR lowering 记录表

层级或 pass	应观察的结构变化	实际文件/结果
arith/func 到 LLVM	llvm.add、llvm.func 和返回值	report_results/ ascendnpusmokey llvm.mlir
AscendNPU HIVM 输入	GM/UB 地址空间、hivm.hir.vadd	report_results/ ascendnpusmokey vecadd_parsed. mlir
HIVM 到 LLVM 方言	LLVM 函数、地址空间指针和 intrinsic	report_results/ ascendnpuhivm_ to_llvm.mlir

5.4 进阶实验的验证边界

只有保存每一步的 IR 文本并用对应工具验证, 才能说明 lowering 过程确实执行。本次已用 bishengir-opt 完成标准 MLIR 解析、arith/func 到 LLVM 以及 HIVM 到 LLVM 的静态转换, 版本与命令记录在 SysY_example/build/report_results/ascendnpusmokeytoolchain_versions.txt。AscendNPU-IR 的安装指南要求匹配的 CANN 工具包和目标环境 [1, 2]; 本次没有配置 CANN 二进制工具包和 Ascend NPU, 因此 bishengir-compile 的设备目标生成与硬件运行仍未验证; 该限制不依赖成员 B 的 RISC-V 运行结果。

6 结果与讨论

6.1 已获得的结果

本次在 WSL 中实际运行 Clang/LLVM 18.1.3，以下检查已完成。结果文件统一保存在 `SysY_example/build/report_results/`：

- 主机参考程序输出 28 3 1；
- `opt -passes=verify SysY_example.ll -disable-output` 通过；
- Clang 成功生成包含四个用户函数和运行库外部声明的 LLVM IR；
- LLVM IR 中能够定位全局对象、局部栈槽、循环条件、数组寻址、算术指令、条件分支和函数调用。

6.2 实验条件与验证边界

目前尚需补充的内容主要是进阶 MLIR/AscendNPU 的设备侧证据。成员 A 的 LLVM IR 结果和成员 B 的 RISC-V 后端结果都已经运行并写入各自章节；直接装配脚本和静态 HIVM lowering 也已当前工具链上运行成功。设备目标生成仍需要匹配版本的 CANN 和 Ascend NPU。

表 6: 实验结果、条件与证据文件

内容	实验状态与限制	证据文件
LLVM IR (成员 A)	已由本次实验实际运行并验证；包括预处理、主机输出、O0/O2、阈值变体和零长度变体。	<code>report_results/</code> 中的 <code>.i</code> 、 <code>.ll</code> 、 <code>.out</code> 、 <code>.diff</code>
RISC-V 后端 (成员 B)	成员 B 的独立报告已记录汇编、直接装配、链接、ELF 检查和模拟器运行；当前工具链输出为 28 3 1。	<code>riscv_from_asm.elf</code> 、 <code>.s</code> 、运行日志
MLIR/AscendNPU 进阶部分	已生成标准 <code>arith/func</code> 到 LLVM 与 HIVM 到 LLVM 的静态结果；CANN/Ascend NPU 未配置。	<code>report_results/ascendnpu_*.mlir</code> 、版本和命令记录

主机输出与静态 IR 分析相互补充：前者说明当前输入上的程序计算结果符合预期，后者说明源程序结构确实被编码为相应的 LLVM IR。单个输入的正确输出不能推出所有输入都正确，因此报告保留阈值、优化级别和零长度输入实验作为进一步检查。

6.3 验证边界

本节聚焦 LLVM IR 的数据布局和生成环境。RISC-V 汇编文本、汇编器命令、目标文件链接和模拟器执行分析见后文，相关代码、日志和分析结果一并列出。

本次运行使用 WSL 中的 Clang/LLVM 18.1.3，以及独立构建的 BiShengIR 2.0.0/LLVM 19.1.7。主机输出、预处理文件、O0/O2 IR、阈值变体、零长度变体、优化差异文件、RISC-V 整链运行日志和 MLIR/AscendNPU 静态 lowering 产物均保存在 `SysY_example/build/report_results/`。由于未配置 CANN 二进制工具包和 Ascend NPU，设备目标生成与硬件运行未纳入本次验证。

7 结论

本实验通过一个覆盖数组、循环、条件、函数和运行库调用的 SysY 示例，观察了预处理器如何向源程序注入运行库声明，以及 Clang 如何将源代码转换为 LLVM IR。分析表明，`alloca`、`load`、`store`

表示局部变量和内存读写, `getelementptr` 表示数组元素地址计算, `icmp` 与 `br` 表示条件和控制流, `call` 与 `ret` 表示函数接口, 外部 `declare` 则保留了运行库链接边界。

通过阈值修改、-O0/-O2 对比和零长度输入实验, 可以把源代码改动与 IR 指令、基本块和保护分支的变化联系起来。成员 A 的 LLVM IR 结果与成员 B 的 RISC-V 结果在后续章节中形成互相独立的证据链。

成员 A 附录：复现实验命令

以下命令均在 lab1/SysY_example 目录执行：

Listing 21: LLVM IR 实验的最小复现命令

```
1 clang --version
2 opt --version
3 llc --version
4
5 clang -E -std=c11 -include sysy_runtime.h \
6     SysY_example.c -o build/SysY_example.i
7
8 clang -target riscv64-unknown-elf -march=rv64gc -mabi=lp64d \
9     -std=c11 -O0 -S -emit-llvm -include sysy_runtime.h \
10    SysY_example.c -o SysY_example.ll
11
12 opt -passes=verify SysY_example.ll -disable-output
```

成员 B 独立报告（RISC-V 与汇编语言）

8 摘要

本实验以成员 A 已生成的 `SysY_example.ll` 和 `SysY_example.riscv.s` 为输入，重点观察 LLVM IR 到 RV64GC 汇编、目标文件和 ELF 可执行文件的后端处理过程。实验首先核对 `llc` 生成的汇编文件，随后通过 RISC-V GCC 将该汇编文本装配为目标文件，再与 `sysy_runtime.c` 链接，最后使用 RV64GC 模拟器运行。分析围绕函数序言、栈帧、寄存器调用约定、数组寻址、分支、除法取余、全局变量寻址和运行库调用展开。

示例程序对数组 `[3, -2, 7, 4, -1]` 乘以 2，求正数和与个数，并输出 `28 3 1`。实验新增了 `build_from_asm.sh`，把“汇编文本-目标文件-链接-模拟器运行”固定为可复现链路，并保存 ELF 信息、反汇编和运行日志。由 C 源程序直接链接的自动脚本和由已生成汇编直接装配的链路都得到相同结果。

关键词：编译器；LLVM IR；RISC-V；RV64GC；汇编器；链接器；ABI；ELF

9 引言

9.1 实验背景

编译器后端负责把目标无关的中间表示降低为具体指令集。LLVM IR 仍然保留类型、函数、基本块和显式控制流等结构，而 RISC-V 汇编需要进一步确定寄存器、栈帧、指令形式、数据段布局和调用约定。汇编器将助记符编码为机器码并生成重定位信息，链接器再解析跨文件符号，把目标文件、启动代码和运行库合成为 ELF。

本实验使用 RV64GC 目标。RISC-V 的整数参数和返回值主要通过 `a0--a7` 传递，`ra` 保存返回地址，`sp` 管理栈顶，`s0` 可作为帧指针。这些 ABI 规则可以在 `SysY_example.riscv.s` 的函数序言、调用点和返回路径中直接观察，并分别对应 RISC-V 指令集规范和 ELF psABI 的寄存器约定 [7, 8]。

9.2 实验范围与分工

本部分覆盖 LLVM IR 生成之后的后端实验：使用 `llc` 得到 RISC-V 汇编，阅读函数和基本块的指令对应关系，使用交叉工具链装配和链接，检查 ELF，并在 RV64GC 模拟器上验证结果。预处理器、LLVM IR 生成、验证和 IR 结构分析见前文；两部分共享同一份 SysY 源程序和运行库接口。

9.3 内容安排

第一部分说明预处理器、编译器前端、LLVM IR、代码生成器、汇编器和链接器的完整工作过程；第二部分改为 RISC-V 与汇编语言实验，记录代码生成、栈帧、寻址、分支、调用约定、装配、链接和运行结果；第三部分说明 MLIR/AscendNPU IR 的逐层 lowering 观察方法；最后给出结果、验证边界、结论和复现命令。

10 第一部分：语言处理系统完整工作过程

10.1 研究对象与阶段总览

成员 B 关注的是“前端已经完成之后，程序怎样变成能够执行的机器码”。因此本部分不重新展开词法和语法规则，而是从产物交接的角度追踪每个阶段：前端把源程序交给 LLVM IR，中端保持语义并整理控制流，后端依据 RV64GC 和 ABI 选择指令，汇编器生成带符号信息的目标文件，链接器再把多个目标模块拼成 ELF。每个阶段都必须满足下一阶段的输入约束，任何一个边界出错都会在后续产物中暴露。

表 7: 从源程序到 RISC-V ELF 的阶段与产物

阶段	主要功能	本实验产物
预处理	注入运行库声明并展开文本指令	<code>.i</code>
前端编译	词法、语法、语义分析并构造 IR	<code>.ll</code>
IR 验证	检查模块、类型和控制流约束	验证返回码
代码生成	根据目标架构和 ABI 选择指令	<code>.riscv.s</code>
汇编	将助记符编码并建立重定位信息	<code>.o</code>
链接	合并目标文件和运行库并解析符号	ELF
运行	加载并执行 RV64GC 机器码	28 3 1

10.2 预处理器做了什么

从后端视角看，预处理器的关键作用是确定最终会出现在 IR 中的外部接口。通过 `-include sysy_runtime.h`，`putint` 和 `putch` 的函数签名在前端阶段已经固定，后端因此可以在汇编中生成一致的调用序列。预处理器本身不会决定寄存器或指令，但它决定了后续模块需要解析哪些外部符号。

10.3 编译器做了什么

前端和中端负责把源程序整理成后端可以消费的 IR，但成员 B 重点观察的是 IR 中哪些信息会在降低过程中被保留或具体化：函数参数变成 ABI 规定的寄存器，局部变量变成栈槽或寄存器，基本块变成标签和跳转，整数运算变成 RV64 的算术指令，外部 `declare` 变成需要链接器解决的符号引用。比如 `getelementptr` 不会以同名指令出现在最终汇编中，而会被展开成元素大小计算、基址加法和具体访存。

10.4 汇编器做了什么

汇编器是汇编语言和二进制目标文件之间的边界。它不仅把 `lw`、`mulw` 等助记符编码成机器码，还要区分代码段和数据段、记录函数标签、生成符号表和重定位项。成员 B 使用独立装配命令，验证的正是 `llc` 输出的文本能否脱离编译器重新进入工具链。

10.5 链接器做了什么

链接器面对的是多个目标文件之间的关系。它需要把汇编模块中的 `putint`、`putch` 外部引用与运行库实现对应起来，同时处理 `positive_count` 的全局地址、代码和数据段布局以及跨文件重定位。成员 B 通过 `readelf` 和 `objdump` 检查链接结果，确认 ELF 不只是“生成了一个文件”，而是包含了可定位、可执行的目标代码。

10.6 端到端过程的验证边界

从后端角度，验证应分为静态和动态两层：静态层检查汇编标签、指令、ELF 段和符号，动态层检查模拟器是否能执行并产生预期输出。仅有 `llc` 输出不能说明 ABI 正确，仅有链接成功也不能说明控制流正确；本实验把 `readelf`、`objdump` 和模拟器输出一起保存，形成从汇编文本到运行结果的闭环。

11 第二部分：RISC-V 与汇编语言实验

11.1 实验环境与文件

实验使用 Clang/LLVM 18.1.3 生成面向 `riscv64-unknown-elf` 的 IR 和 RV64GC 汇编，使用仓库中的 RISC-V GCC 15.1.0、`readelf`、`objdump` 和 `riscv64-unknown-elf-run` 完成后端验证。主要

文件如下：

表 8: 成员 B 后端实验文件

文件	作用
SysY_example.ll	面向 RISC-V 的 LLVM IR 输入
SysY_example.riscv.s	llc 生成的 RV64GC 汇编
sysy_runtime.c/.h	putint/putch 运行库适配层
build_from_asm.sh	直接装配、链接、检查和运行脚本
build/report_results/	ELF、反汇编、版本和运行日志

11.2 生成并检查 RISC-V 汇编

先验证成员 A 生成的 IR，再使用 llc 指定 RISC-V 三元组和 m,a,f,d,c 扩展生成汇编：

Listing 22: LLVM IR 到 RISC-V 汇编

```
1 opt -passes=verify SysY_example.ll -disable-output
2 llc -mtriple=riscv64-unknown-elf \
3   -mattr=+m,+a,+f,+d,+c -filetype=asm \
4   SysY_example.ll -o SysY_example.riscv.s
5 grep -E '^(scale_array|sum_positive|classify_average|main):' \
6   SysY_example.riscv.s
```

汇编文件包含四个用户函数。函数标签、.text 段、全局对象的 .rodata/.sbss 段以及运行库调用共同构成了可供汇编器处理的目标文本。

11.3 函数序言、栈帧和 ABI

scale_array 的典型序言为：

Listing 23: scale_array 的函数序言

```
1 addi sp, sp, -48
2 sd   ra, 40(sp)
3 sd   s0, 32(sp)
4 addi s0, sp, 48
```

函数分配 48 字节栈帧，保存返回地址和帧指针。数组首地址、长度、缩放因子和循环变量分别放在 -24(s0)、-28(s0)、-32(s0) 和 -36(s0)。函数结束时用 ld 恢复寄存器、用 addi 回收栈空间并执行 ret。在 main 中，编译器还保存 s1 来复用全局变量地址，因此其栈帧为 64 字节。

11.4 数组寻址与内存访问

数组访问 values[i] 在汇编中表现为：

Listing 24: 的 RISC-V 地址计算]values[i] 的 RISC-V 地址计算

```
1 lw   a1, -36(s0)
2 slli a1, a1, 2
3 add  a0, a0, a1
4 lw   a1, 0(a0)
```

由于 `int` 占 4 字节，`slli` 左移 2 位得到字节偏移，`add` 把偏移加到数组首地址，`lw` 再取出元素。写回时把最后一条替换为 `sw`。这对应 LLVM IR 中的 `sext`、`getelementptr`、`load` 和 `store`，但汇编已经把地址计算具体化为 RV64 指令。

11.5 循环、分支与算术指令

`scale_array` 的循环头先用 `lw` 取出 `i` 和 `length`，再用 `bge` 判断循环是否结束；循环体用 `mulw` 完成 32 位乘法，使用 `addiw` 将下标加一。`sum_positive` 使用 `blez` 跳过非正元素。循环和条件分支的标签 `.LBB0_1`、`.LBB1_1` 等对应 IR 的基本块和回边。

`classify_average` 先用 `bnez` 检查 `count` 是否为零，然后分别用 `divw` 和 `remw` 实现整数除法和取余；后续 `blt`、`bltz` 完成阈值和符号判断。由此可见，源程序中的 `count == 0` 保护被保留为真实控制流，而不是由模拟器或链接器补救。

11.6 函数调用和运行库接口

RV64GC ABI 使用 `a0`、`a1`、`a2` 传递 `scale_array` 的三个参数，函数返回值也放在 `a0`。主程序在调用 `putint` 前把待输出整数放入 `a0`，在调用 `putch` 前放入空格 ASCII 码 32 或换行 ASCII 码 10。汇编中的 `call putint` 和 `call putch` 是对外部运行库符号的引用，实际地址由链接器确定。

11.7 汇编、链接与运行

直接装配链路由 `build_from_asm.sh` 执行：

Listing 25: 直接装配已生成汇编并运行

```
1 bash build_from_asm.sh
```

脚本先检查汇编中存在 `main`、`putint` 和 `putch`，再执行：

Listing 26: 汇编、链接和模拟器命令

```
1 riscv64-unknown-elf-gcc -march=rv64gc -mabi=lp64d \  
2 -c SysY_example.riscv.s -o build/SysY_example.from_asm.riscv.o \  
3 riscv64-unknown-elf-gcc -march=rv64gc -mabi=lp64d \  
4 -specs=sim.specs build/SysY_example.from_asm.riscv.o \  
5 sysy_runtime.c -o build/SysY_example.from_asm.elf \  
6 riscv64-unknown-elf-readelf -h -S -s \  
7 build/SysY_example.from_asm.elf \  
8 riscv64-unknown-elf-run --model RV64GC \  
9 build/SysY_example.from_asm.elf
```

日志、ELF 信息、反汇编和输出分别保存为以下文件：

- `riscv_from_asm.log`;
- `riscv_from_asm.elf`;
- `riscv_from_asm.elf_info.txt`;
- `riscv_from_asm.disassembly.txt`;
- `riscv_from_asm.output.txt`。

预期运行结果为：

Listing 27: 直接装配链路输出

```
1 28 3 1
```

11.8 实验结果与分析

主机参考程序、自动 RISC-V 链接脚本和直接装配生成的 ELF 都应输出 28 3 1。前两者验证源程序语义和目标运行库链路，后者进一步验证了 `SysY_example.riscv.s` 的汇编、重定位、外部符号解析和目标执行。对 `objdump -d` 输出进行检查时，可以重新定位四个函数的序言、数组下标偏移、`divw/remw` 和 `call` 指令，因此静态指令分析与动态输出形成相互支撑。

12 第三部分：MLIR 中端处理过程（进阶要求）

12.1 任务目标与层级组织

成员 B 从目标代码生成的角度观察 MLIR：一个高层的 `VecAdd` 操作什么时候变成显式循环，什么时候变成连续内存访问，什么时候才允许目标后端决定向量宽度、地址空间和硬件布局。LLVM IR 往往已经把很多高层信息压低，而 MLIR 通过多方言保留中间层语义，使后端可以在更合适的时机进行循环变换和硬件映射。

12.2 `VecAdd` 的逐层 lowering 观察点

1. 在高层方言中确认两个输入和一个输出的形状、元素类型及并行迭代语义；
2. 在循环方言中检查循环边界是否仍为完整的向量长度，以及边界条件是否改变；
3. 在向量方言中观察一次处理多少个元素、读写是否连续，以及尾部元素如何处理；
4. 在 LLVM 方言中检查指针、`load/store`、函数签名和控制流是否已经满足后端接口；
5. 在 `AscendNPU` 或其他目标方言中确认设备内存、核函数和布局约束是否被加入。

12.3 命令与结果记录

本次已在 WSL Ubuntu 24.04 中安装可运行的 `bishengir-opt` 和 `bishengir-compile`，并为每一个已执行的 lowering 保存输入、输出和版本信息。当前记录覆盖标准 `arith/func` 到 LLVM 以及 `AscendNPU HVM` 到 LLVM；循环到向量的完整课程 `VecAdd` 管线和设备侧硬件映射仍需 `CANN` 与 `Ascend NPU`：

Listing 28: MLIR/AscendNPU lowering 实际执行命令

```
1 bishengir-opt --version
2 bishengir-opt ascendnpu_smoke_input.mlir \
3   --convert-arith-to-llvm \
4   --convert-func-to-llvm \
5   -o ascendnpu_smoke_llvm.mlir
6 bishengir-opt ascendnpu_hvm_conversion_input.mlir \
7   --split-input-file \
8   --convert-hvm-to-llvm \
9   -o ascendnpu_hvm_to_llvm.mlir
```

表 9: MLIR lowering 记录

层级	应观察的结构变化	当前状态
arith/func 到 LLVM	llvm.add、llvm.func 和返回值	report_results/ ascendnpusmoke_ llvm.mlir
AscendNPU HIVM 输入	GM/UB 地址空间、hivm.hir.vadd	report_results/ ascendnpusmoke_ vecadd_parsed. mlir
HIVM 到 LLVM 方言	LLVM 函数、地址空间指针和 intrinsic	report_results/ ascendnpusmoke_ to_llvm.mlir

12.4 进阶实验的验证边界

本次保存的每一步 MLIR 文本和版本记录可以证明上述静态 pass 已实际执行；但没有 CANN 二进制工具包和 Ascend NPU，无法确认设备目标生成、硬件运行或性能数据。因此本节不把推测的 AscendNPU 汇编写入报告。这与本实验已经完成的 RV64GC 汇编、目标文件、ELF 和模拟器验证是两条不同的证据链。

13 结果与讨论

13.1 已获得的结果

从成员 B 的后端任务看，本实验获得以下可检查结果：

- 目标三元组和 RV64GC 扩展被正确传递给 llc，生成四个用户函数；
- 汇编中的栈帧、寄存器、访存和分支可以与源程序的运行行为对应；
- build_from_asm.sh 将汇编文本作为真实输入，而不是重新编译 C 源；
- ELF 信息和反汇编记录了链接后的机器架构、段、符号和函数代码；
- 直接装配生成的目标程序在 RV64GC 模拟器上输出 28 3 1。

13.2 结果解释与验证边界

直接装配链路的成功意味着汇编器能够识别 llc 产生的语法，链接器能够解析运行库和全局符号，模拟器能够执行其中的 RV64GC 指令。静态指令序列解释了输出一致的原因：数组更新使用 mulw 和 sw，正数筛选使用 blez，分类函数使用 divw/remw 和保护分支，输出则遵循 a0 参数约定。单一输入仍不足以覆盖所有寄存器压力、溢出和边界长度，所以结论限定在当前示例和已保存的变体实验内。

直接装配链路与自动脚本存在验证边界差异：自动脚本的目标 ELF 由 C 源重新编译生成，而 build_from_asm.sh 明确使用 SysY_example.riscv.s 作为汇编输入。后者才可以证明 llc 输出本身已经进入最终执行路径。

14 结论

从成员 B 的角度，本实验最重要的认识是：汇编语言不是 LLVM IR 的简单文本替换，而是对栈布局、寄存器约定、内存宽度、跳转目标和外部符号的一次具体决策。slli 与 add 落实数组寻址，

`bge/blez/ bnez` 落实控制流, `mulw/divw/remw` 落实整数运算, `call` 和重定位则连接了汇编文件与运行库。

通过直接装配、ELF 检查、反汇编和 RV64GC 模拟器运行, 成员 B 验证了已生成汇编能够独立进入最终执行链路。MLIR/AscendNPU 部分已完成工具安装和 HVM→LLVM 静态流程, 本实验未配置 CANN 二进制工具包和 Ascend NPU, 因此未生成设备目标, 也未进行硬件运行验证。

成员 B 附录：复现实验命令

以下命令均在 lab1/SysY_example 目录执行：

Listing 29: RISC-V 后端实验的最小复现命令

```
1 opt -passes=verify SysY_example.ll -disable-output
2 llc -mtriple=riscv64-unknown-elf \
3   -mattr=+m,+a,+f,+d,+c -filetype=asm \
4   SysY_example.ll -o SysY_example.riscv.s
5 bash build_from_asm.sh
6 cat build/report_results/riscv_from_asm.output.txt
```

参考文献

References

- [1] Ascend. Ascendnpu-ir: Build and installation. https://github.com/Ascend/AscendNPU-IR/blob/master/docs/source/en/introduction/quick_start/installing_guide.md, 2026. Online documentation, accessed 2026-10-01.
- [2] Huawei Ascend Community. Cann community edition documentation. <https://www.hiascend.com/document/detail/en/CANNCommunityEdition/900/index/index.html>, 2026. Online documentation, accessed 2026-10-01.
- [3] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. Mlir: Scaling compiler infrastructure for domain specific computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 2–14. IEEE, 2021.
- [4] LLVM Project. Clang command-line reference. <https://clang.llvm.org/docs/ClangCommandLineReference.html>, 2024. Clang documentation.
- [5] LLVM Project. Llvm language reference manual. <https://releases.llvm.org/18.1.3/docs/LangRef.html>, 2024. LLVM 18.1.3 documentation.
- [6] Nankai University Compiler Systems Course Group. Sysy 2022 language definition, version 1, 2022. 课程资料，实验目录中的本地 PDF.
- [7] RISC-V International. The risc-v instruction set manual, volume i: Unprivileged architecture. <https://docs.riscv.org/reference/isa/v20240411/unpriv/unpriv-index.html>, 2024. Version 20240411, ratified specification.
- [8] RISC-V Non-ISA Specifications. Risc-v elf psabi document. <https://riscv-non-isa.github.io/riscv-elf-psabi-doc/>, 2026. Online specification, accessed 2026-10-01.